Name-Free Wire Connectivity for Quantum-Circuit Learning: Exact Relabeling Invariance and Width Extrapolation

Anonymous Authors
Affiliation withheld for double-blind review

***Abstract*—Learned quantum-circuit models are usually trained on circuits that are smaller than the circuits they are meant to handle. Grid and sequence encodings expose a gate's layer, token index, or canonical position, and these positions are tied to the training size and to the arbitrary names of logical qubits. We encode a circuit as a wire-dependency directed acyclic graph (DAG) whose nodes carry only gate type, angle, and arity and whose edges record the operand roles (single-qubit, control, or target) of the two operations on their shared wire. No feature names a qubit, so per-gate predictions are exactly invariant to any permutation of logical-qubit labels. We test the encoding on an exactly labeled probe: predicting which gates lie on a maximum dependency path. Models were trained on 4–7-qubit QAOA-like, VQE, and arithmetic circuits under criteria frozen before the final runs. On 13–20-qubit circuits from the same families, the 80,833-parameter DAG reaches AUROC 0.9252 against 0.8807 for an 84,961-parameter canonical-grid GRU that sees no connectivity, winning all five paired seeds (mean $+0.0444$), and its logits do not change at all under relabeling. Pre-registered post-confirmatory controls then locate the source of the gain. A GRU given name-free predecessor and successor pointers on each wire ties the DAG at 13–20 qubits (0.9274; paired difference -0.0023, interval $[-0.0083, +0.0038]$) but loses 0.0615 AUROC when qubits are renamed. Removing the DAG's operand roles costs only 0.0053. The width gain therefore comes from exposing wire connectivity without qubit names; what the graph adds is exact relabeling invariance by construction. A held-out chemistry-inspired family and three downstream compiler tasks did not favor the DAG.**

***Index Terms*—quantum circuits, graph neural networks, permutation invariance, out-of-distribution generalization, size generalization, critical path**

I. Introduction

Learning-based tools for quantum circuits face a built-in size mismatch. Labels, simulation, and compiler search are cheapest on small circuits, but the circuits of interest are larger. A learned circuit encoder therefore has to *extrapolate* in width and depth, not just interpolate between familiar shapes.

Many encoders serialize a circuit as a grid or token sequence. Each gate then has a position: its layer, its token index, or its slot in a canonical order. On familiar sizes these positions correlate with structural labels, but their meaning drifts when qubits and gates are added. The same holds when logical qubits are renamed, since a canonical order must break ties using wire names. Fig. 1(d) shows the effect on a three-qubit circuit. Swapping the names q_0 and q_2 changes the sequence positions of four of the six gates, even though the circuit is the same computation up to naming.

Other relations keep their meaning at every size and under every renaming. One example is "this CX is the next operation, in its target role, after an RZ on the same wire." The wire-dependency DAG is built from exactly these relations. We encode them with *operand roles* on the edges instead of absolute wire identifiers. Relabeling the qubits then leaves the input graph unchanged (Fig. 1(c)), and the model is invariant by construction rather than by augmentation.

We ask three questions. (Q1) Can a dependency-graph encoder enforce exact invariance to logical-qubit relabeling? (Q2) Does it extrapolate better from small to much wider circuits within the same families? (Q3) If so, is the gain due to graph message passing, or to exposing which operations share a wire? We probe both with *maximum dependency-path membership*: label each gate by whether it lies on a longest path of the dependency DAG. The labels are exactly computable, so the probe has no simulator or label noise. The task isolates whether a representation supports multi-hop dependency reasoning. We do not propose the network as a replacement for the linear-time exact algorithm.

Contributions. (i) An index-free circuit representation whose per-gate predictions are provably invariant to logical-qubit permutations (Proposition 1). (ii) A five-seed out-of-distribution benchmark with success criteria frozen before the final runs. Models train at 4–7 qubits and are tested at up to 20 qubits. The DAG beats a near-matched canonical-grid GRU without connectivity on every seed. (iii) Pre-registered connectivity controls, whose protocol was committed before any code or result existed. They show that a sequence model with name-free wire pointers matches the DAG's width extrapolation but not its relabeling invariance, and that operand roles contribute little. The width gain is about *exposing connectivity without names*, not about graphs as such. (iv) Supporting earlier evidence that width growth, not depth growth, accounts for most of the advantage, and negative results that limit the claim to within-family structural extrapolation.

II. Related Work

Circuit representations. Graph encoders for circuits treat gates as nodes and wires as edges. Examples include QuEst, which predicts circuit reliability with a graph Transformer [1], and LayerDAG-based generators. LayerDAG generates generic DAGs layer by layer with diffusion [2]. Q-Fusion adapts it to generate quantum circuits and adds learned representations

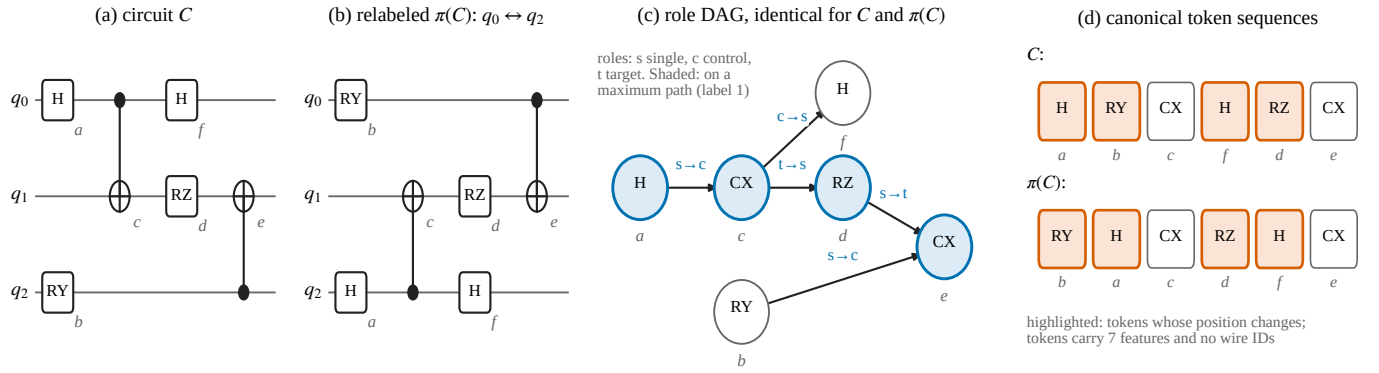


Fig. 1. One circuit under two logical-qubit labelings, computed with the benchmark's own code. (a,b) Swapping the names $q_0 \leftrightarrow q_2$ is a symmetry of every structural property. (c) The role-labeled dependency DAG, with edge attribute (source role $\rightarrow$ destination role), is identical for C and $\pi(C)$, including the maximum-path labels. (d) The canonical earliest-layer order used by the sequence baselines breaks ties with wire indices, so four of six tokens move. Italic letters mark the same operation across panels.

of wire connections, analogous to node-type embeddings [3]. Any categorical vocabulary indexed by absolute wires or ordered wire pairs is defined relative to a fixed enumeration of qubits. It grows with the supported width and is not, by itself, invariant to renaming. Our encoder replaces such identities with local operand roles. This identifies a possible limitation under relabeling and width shift. It does not challenge the results reported for those methods on their own tasks and distributions. Our implementation borrows LayerDAG-style sparse bidirectional aggregation but does not generate graphs.

Learned compilation. Learning has been used to optimize circuits [4] and to support routing and mapping, which are classical compiler problems [5]–[7]. We study a representation question that comes before such uses and make no compiler claim.

Symmetry and extrapolation. Matching architectural symmetry to task symmetry is a central principle of geometric deep learning [8]–[10]. Message-passing networks [11], [12] can learn to execute graph algorithms [13]. Their extrapolation to larger graphs depends on local structure and on how well the architecture is aligned with the algorithm [14], [15]. Our probe is a longest-path computation, which places it in this algorithmic-alignment setting.

III. Representation and Invariance

A. Wire-dependency DAG with operand roles

A circuit $C = (o_1, \ldots, o_m)$ on logical qubits Q is an ordered list of operations $o_i = (g_i, \mathbf{w}_i, \theta_i)$. Here $g_i \in \{\mathrm{H}, \mathrm{RY}, \mathrm{RZ}, \mathrm{CX}\}$, $\mathbf{w}_i$ is a tuple of distinct qubits (control first for CX), and θ_i is an angle (0 if absent). For each $q \in Q$, let $i_1 < \cdots < i_k$ index the operations that act on q. We add a directed edge $(i_j \rightarrow i_{j+1})$ for each consecutive pair. The union over all wires is the dependency DAG $G(C)$; independent operations are never connected. The role of qubit q in operation i is $r(i, q) \in \{\mathrm{s}, \mathrm{c}, \mathrm{t}\}$ (single-qubit operand, CX control, or CX target). The edge $(i_j \rightarrow i_{j+1})$ induced by q carries the six-dimensional attribute $[\mathrm{onehot}(r(i_j, q)), \mathrm{onehot}(r(i_{j+1}, q))]$. Each node carries seven features: a four-way gate one-hot, $\sin\theta_i$, $\cos\theta_i$, and arity$/2$. No feature names a wire.

The network has six residual blocks. Each block applies separate linear messages to (sender state $\|$ edge roles) along forward and backward edges, sums them with `index_add`, and applies a self transform, a GELU, and LayerNorm. A linear head outputs one logit per gate. The model has 80,833 parameters.

B. Exact invariance

Proposition 1. *Let π be any permutation of Q and $\pi(C) = (g_i, \pi(\mathbf{w}_i), \theta_i)_{i=1}^m$. Then $G(\pi(C))$ and $G(C)$ have identical node features and identical multisets of attributed edges. Consequently, any model that consumes the node features and an unordered multiset of attributed edges receives identical input for C and $\pi(C)$, and so satisfies $f(\pi(C))_i = f(C)_i$ for every i. The sum-aggregation network above is such a model. The labels satisfy $y(\pi(C)) = y(C)$.*

Proof. Node features do not depend on $\mathbf{w}_i$. Operation i acts on $\pi(q)$ in $\pi(C)$ exactly when it acts on q in C, so the ordered chain of operations on wire $\pi(q)$ equals the chain on q. Because π is applied entrywise, q's position in $\mathbf{w}_i$ equals $\pi(q)$'s position in $\pi(\mathbf{w}_i)$, so $r(i, \pi(q)) = r(i, q)$. The edges contributed by wire $\pi(q)$ in $\pi(C)$ are therefore exactly those contributed by q in C, with the same endpoints and attributes. Because no attribute records the wire, the two edge multisets coincide. Sum aggregation does not depend on edge order, so every layer, and hence every logit, agrees. The labels depend only on the edge structure. □

Because the model is permutation-equivariant over nodes, its predictions also follow any legal reordering of commuting operations. The guarantee is exact in exact arithmetic. In floating point, summing edges in a different order could perturb the last bits, so the protocol tolerance was set at 10^{-6}.

C. Sequence baselines and what each model receives

Both baselines receive the same seven index-free node features. Gates are sorted by the key (earliest dependency

TABLE I
INPUTS AND PROPERTIES OF THE COMPARED ENCODERS

	Role DAG	Grid GRU	Transf.	Ptr. GRU
Gate, $\sin\theta$, $\cos\theta$, arity	yes	yes	yes	yes
Absolute wire identifiers	no	no	no	no
Shared-wire connectivity	edges	implicit	implicit	pointers
Operand roles	on edges	no	no	per token
Order depends on names	no	tie-breaks	tie-breaks	tie-breaks
Exactly invariant	yes	no	no	no
Parameters	80,833	84,961	88,065	87,457
Evidence tier	primary	primary	post	post

layer, wire tuple, gate, angle). The *canonical-grid GRU* [16] (two bidirectional layers, width 96, 84,961 parameters) reads this sequence. The *canonical Transformer* [17] (width 64, two layers, four heads, feed-forward width 160, learned positions for up to 192 tokens, no dropout, 88,065 parameters) reads the same sequence. Table I states exactly what each model sees. The sequence models get no wire identifiers, but they also get no explicit connectivity. Shared-wire structure reaches them only through the canonical order, and wire names enter that order only as a tie-breaker. The primary comparison is therefore between an index-free encoding that *exposes* dependency relations and index-free serializations that *do not*.

Connectivity controls. To separate connectivity from graph structure, a *pointer GRU* (87,457 parameters) reads the same canonical sequence with 26 extra features per token. For each operand (control or single-qubit operand first, then target) it receives the operand's role and, for the previous and the next operation on the same wire, an existence flag, that operation's role, and the log canonical-position distance. Wire names are used only to find these neighbors and are never emitted. Roles and existence flags are invariant to relabeling, but the distances are measured in the canonical order, whose tie-breaks depend on names, so the model is not invariant. A *pointer Transformer* (89,729 parameters) uses the same features with the Transformer configuration above. A *role-ablated DAG* sets every edge attribute to zero, keeping topology only. Section V-E reports earlier experiments in which both model types saw absolute wire identities.

D. Probe task

Let $a(v)$ be the number of operations on the longest dependency path ending at v, $b(v)$ the number on the longest path starting at v, and $D = \max_v a(v)$. The label is $y_v = \mathbf{1}[a(v) + b(v) - 1 = D]$, meaning v lies on at least one maximum path. It is computed exactly in linear time. Positive rates are 0.700, 0.568, and 0.517 on the same-size, near-width, and far-width test sets.

IV. EXPERIMENTAL PROTOCOL

Data. Circuits come from three seeded generators with equal shares. QAOA-like circuits consist of a Hadamard layer followed by alternating CX–RZ–CX cost blocks and H–RZ–H mixer blocks [18]. Hardware-efficient VQE circuits [19] use RY/RZ layers and cyclically offset CX entanglers. Reversible linear arithmetic uses pivot-controlled CX fans with RZ phases. Training uses 1,000 circuits with 4–7 qubits and 32–64 gates; validation uses 256 fresh circuits. Each of four final splits has 512 circuits: *same size* (4–7 qubits, 32–64 gates), *near width* (8–12, 72–104), *far width* (13–20, 96–144), and *relabeled* (the same-size circuits after an independent random qubit permutation per circuit). Content hashes of all splits are stored with the results. Regenerating the splits from their seeds reproduces all six hashes, and no test circuit duplicates a training circuit.

Training. Each model trains for 4,000 AdamW updates (learning rate 3×10^{-4}, batch 16, clipping 1.0, positive-class weighting) under paired seeds 111–115. A given seed fixes the initialization stream and the minibatch indices for both architectures. The checkpoint is chosen by validation AUROC, evaluated every 250 updates. The final splits are never used for training or selection.

Frozen gate. A development run on separate data (one seed, 2,000 updates) fixed the architectures and size ranges. According to the project records, the final protocol then fixed the following success criteria before the final data were generated (see the provenance note in Section VI). (1) The DAG's far-width AUROC exceeds the grid's in all five seeds. (2) The mean improvement is at least $+0.03$. (3) The DAG's mean far-width F1 exceeds the grid's. (4) The maximum change in DAG logits under relabeling is at most 10^{-6}. (5) The DAG's same-size metrics are unchanged by relabeling.

Evidence tiers. Primary confirmatory: the five-seed DAG-versus-grid final. *Post-confirmatory controls:* the Transformer, whose configuration was fixed after the primary result had been opened, and the three connectivity controls, whose protocol, including pass/fail readings, was committed and pushed to the project repository before their code was written. All use the same data and seeds and received no tuning. *Supporting:* two earlier frozen studies with a different, wire-indexed architecture (Section V-E). We never pool statistics across tiers.

Statistics. We report node-level AUROC and F1 (threshold 0.5), with means and sample standard deviations over the five training seeds. Each seed's DAG and grid share data and minibatches, so we report per-seed paired differences and a descriptive Student-t interval over seeds ($\mathrm{df} = 4$). The test sets are fixed, so this interval covers training randomness only, not sampling of circuits. Gates within a circuit are not independent, and we do not treat them as replicates.

V. RESULTS

A. Primary result: width extrapolation

All five gate conditions pass (Table II, Fig. 2). The paired far-width AUROC differences (DAG minus grid) are $+0.0419$, $+0.0419$, $+0.0417$, $+0.0482$, and $+0.0484$ for seeds 111–115. Their mean is $+0.0444$ (s.d. 0.0035), with a descriptive seed-level 95% t interval of $[+0.0400, +0.0488]$. The DAG's far-width F1 is 0.8458 against 0.8011, a gain of $+0.0446$. The DAG also leads at the same size ($+0.0331$ AUROC) and at

TABLE II
FROZEN FIVE-SEED FINAL: AUROC (MEAN ± S.D.) AND F1

	Role DAG (primary)		Grid GRU (primary)		Transf.†
Split	AUROC	F1	AUROC	F1	AUROC
Same, 4–7 q	.9722±.0009	.9383	.9391±.0028	.8919	.8888±.0010
Near, 8–12 q	.9362±.0008	.8749	.8853±.0027	.8034	.7467±.0070
Far, 13–20 q	.9252±.0020	.8458	.8807±.0045	.8011	.7471±.0128
Relabeled	.9722±.0009	.9383	.8294±.0053	.8199	.7806±.0064

†Post-confirmatory descriptive control, not part of the frozen gate. Its F1 (same/near/far/relabeled) is .8543/.7427/.7263/.8064.

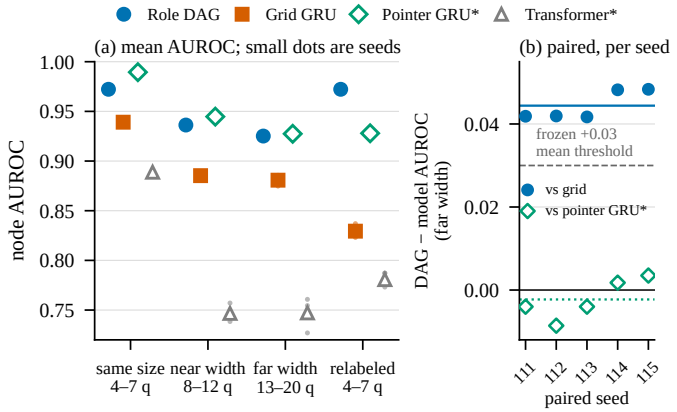


Fig. 2. (a) Mean AUROC per split, with individual seeds as small dots. Open markers (*) are post-confirmatory controls. (b) Paired far-width differences, DAG minus model. Against the grid (filled) all five seeds clear zero and the mean clears the frozen +0.03 threshold; against the pointer GRU (open) the differences straddle zero.

near width (+0.0509). The advantage is therefore not specific to the extrapolation regime, but it holds on circuits up to 20 qubits, roughly three times the training width.

B. Logical-qubit relabeling

On the 64 circuit–relabeling pairs checked per seed (320 in total), the largest change in any DAG logit is exactly 0.0. The DAG's AUROC, F1, and accuracy are identical on the original and relabeled same-size splits. A post-hoc re-run of seed 111 (DAG and grid) reproduced every recorded split AUROC bit for bit, and the re-run DAG's maximum logit change over all 512 relabeled circuits was also 0.0. The grid's AUROC drops from 0.9391 to 0.8294 (−0.1097), and the Transformer's from 0.8888 to 0.7806. Removing explicit wire indices from a serialization therefore does not make it invariant. The computation itself has to respect the symmetry. Neither baseline was trained with relabeling augmentation. Augmentation could reduce their drop, but it would give at best approximate invariance, learned only on the training sizes.

C. Post-confirmatory Transformer control

The Transformer is weaker than both primary models on every split (far width 0.7471). Its near- and far-width scores are nearly equal, so its loss happens mostly between the training and near-width regimes. This control shows that the conclusion

TABLE III
CONNECTIVITY CONTROLS (POST-CONFIRMATORY): AUROC (MEAN ± S.D.) AND DAG−MODEL FAR-WIDTH PAIRED DIFFERENCES

Split	Role DAG	Pointer GRU	Pointer Transf.	Role-ablated DAG
Same, 4–7 q	.9722±.0009	.9895±.0010	.9481±.0019	.9670±.0013
Near, 8–12 q	.9362±.0008	.9447±.0026	.7533±.0168	.9282±.0019
Far, 13–20 q	.9252±.0020	.9274±.0050	.6900±.0192	.9199±.0025
Relabeled	.9722±.0009	.9279±.0044	.8838±.0035	.9670±.0013
DAG−model, far	—	−.0023	+.2352	+.0053
DAG wins (of 5)	—	2	5	5

Role DAG values are the recorded primary results. A re-run of seed 112 in the controls' software environment reproduced every recorded DAG and grid AUROC bit for bit.

does not hinge on a recurrent baseline. Because the control was added after the primary result was known and was not tuned, it is not confirmatory. A tuned attention model could do better.

D. Post-confirmatory connectivity controls

Table III answers Q3. The protocol fixed three readings for the pointer GRU before any code existed: *topology supported* if the DAG wins all five seeds by a mean of at least +0.02; *information explains the gap* if the mean is below +0.01 or the pointer GRU wins at least three seeds; intermediate otherwise. The pointer GRU wins three seeds and the paired far-width difference is −0.0023 (s.d. 0.0049, interval $[-0.0083, +0.0038]$), so the pre-declared reading is *information explains the gap*. Given name-free wire pointers, a plain bidirectional GRU matches the DAG at 13–20 qubits and beats it at the training size (0.9895). It exceeds the canonical grid by +0.0467 at far width, about the size of the DAG's own margin. At the fixed 0.5 threshold the DAG keeps a higher far-width F1 (0.8458 against 0.8081, all five seeds), so the two models differ in calibration under shift, but this was not a pre-declared criterion. The pointer GRU is not invariant: its AUROC falls by 0.0615 under relabeling, to 0.9279 against the DAG's unchanged 0.9722.

The pointer Transformer also improves at the training size (0.9481) but collapses with width (0.6900), probably because its learned absolute positions never see sequences this long; relative positions were not tested. Removing operand roles from the DAG lowers far-width AUROC by 0.0053 (interval $[+0.0019, +0.0086]$, all five seeds), below the protocol's 0.01 threshold, so roles are *not needed* for this probe by the pre-declared rule even though their small effect is consistent.

E. Supporting evidence: width versus depth

Two earlier frozen studies used a different architecture. In both, the DAG and the grid received *absolute* wire-membership, control, and target one-hots (at most 12 wires), so both had full connectivity information. Neither study is pooled with the primary result.

Structured-family decomposition (Table IV). This study used the same three families with 87,745-parameter (DAG) and 88,417-parameter (grid) models. It separated the two shift axes. Adding wires produced a +0.0514 advantage, whereas adding depth alone produced +0.0025. Most of the structural

TABLE IV
SUPPORTING STRUCTURED STUDY (WIRE-INDEXED ENCODERS, SEEDS 81–85): MEAN AUROC

Split (qubits; gates)	DAG	Grid	Δ	DAG wins
Same size (4–7; 32–64)	.9677	.9623	$+.0054$	5/5
Width only (8–12; 32–64)	.8922	.8408	$+.0514$	5/5
Depth only (4–7; 72–104)	.9589	.9564	$+.0025$	5/5
Width and depth (8–12; 72–104)	.8902	.8495	$+.0407$	5/5
Held-out chemistry (8–12; 72–104)	.5571	.5837	$-.0266$	0/5

advantage therefore comes from width growth. This finding motivated removing wire identifiers so that evaluation could go beyond 12 qubits. Both models had absolute wire identities here, yet the DAG still gained with width. Together with Section V-D, this suggests that what transfers is *local, name-free* neighbor information: absolute identities carry connectivity only through names that do not generalize, whereas dependency edges and relative pointers expose it directly. The comparison is suggestive, because the architectures and the width range differ from the primary study.

Random-circuit reversal. An earlier three-seed study on random circuits (trained on 4–8 qubits) found a reversal. The grid was clearly better at the training size (0.9406 against 0.8556 AUROC). The DAG was better on 9–12-qubit, 64–96-gate circuits (0.8078 against 0.7861), winning all three seeds by $+0.0177$ to $+0.0274$. Validation at the training size alone would have selected the representation that transfers worse.

VI. LIMITATIONS AND NEGATIVE RESULTS

Scope of the probe. Maximum-path membership is computed exactly by a linear-time algorithm. The task tests representational inductive bias, not usefulness, and a compiler should compute critical paths directly. All circuits are synthetic, use four gate types, and are noise-free. Nothing is executed on hardware.

Unseen families. On a held-out chemistry-inspired family, both models were near chance and the grid led by 0.0266 AUROC (Table IV). Within-family width extrapolation does not imply family transfer.

Downstream tasks. Three compiler-facing follow-ups did not establish a DAG advantage. First, in small-angle deletion ranking, a simple heuristic saturated the task. Second, in safe identity-rewrite prioritization, the DAG was favored in all three final seeds, but its $+0.0379$ oracle-recovery gain missed the frozen $+0.05$ threshold, so that gate failed. Third, in routed-rewrite selection, the development runs favored the grid (6.30 against 6.66 routed CX gates removed on average), and no final test was opened. In a separate target-conditioned state-preparation study, a DAG generator was shown to use its conditioning. However, only 9 of 128 targets reached fidelity 0.99, whereas an exact compiler is essentially exact. We claim no compiler or generation advantage.

What the DAG does not uniquely provide. On this probe, width extrapolation is matched by a sequence model with name-free wire pointers, so the primary gain over the canonical grid should not be attributed to graph message passing. The DAG's distinct benefit here is exact relabeling invariance at no accuracy cost. A sequence model trained with relabeling augmentation could narrow that gap approximately; we did not test this.

Statistical and design limits. Five seeds on fixed test sets bound training variance only. No model was tuned, and each Transformer received one fixed configuration; relative-position attention and symmetry-aware graph Transformers remain untested. The models are near-matched (80,833 to 89,729 parameters), not exactly matched.

Provenance. The protocols, code, and results are archived with dataset hashes, per-seed training histories, and a compute ledger (a lower bound of 237 minutes of recorded wall-clock CPU time; no GPU). According to the project records, each primary and supporting protocol was written before its final run. However, those protocols were committed in the same commit as their results, so the repository alone does not timestamp them independently. The connectivity-control protocol was committed and pushed before its code and results, and records one pre-implementation correction: its text called every pointer feature relabeling-invariant, which is false for the distance features. The implementation follows LayerDAG's design in native PyTorch and claims no runtime parity with the upstream DGL code.

VII. CONCLUSION

Operand-role dependency graphs encode the symmetry of logical-qubit relabeling by construction. In a frozen experiment, models trained on 4–7-qubit QAOA-like, VQE, and arithmetic circuits were tested on 13–20-qubit circuits from the same families. The role DAG beat a near-matched canonical-grid GRU on all five paired seeds (AUROC 0.9252 against 0.8807). Pre-registered controls show where this gain comes from: a GRU given name-free predecessor and successor pointers matches the DAG at width, and operand roles add little. Exposing wire connectivity without qubit names is what drives width extrapolation on this probe. The dependency graph is one clean way to do so, and the only encoder tested that is also exactly invariant to relabeling; the pointer GRU loses 0.0615 AUROC when qubits are renamed. The results do not establish graph superiority, robustness to unseen families, or a practical compiler advantage.

ACKNOWLEDGMENT

Funding and acknowledgments are withheld for double-blind review. In accordance with IEEE policy on AI-generated content, the authors disclose that a large language model (Anthropic Claude) was used to edit and restructure the manuscript text, to generate figure-plotting code, to implement and run the post-confirmatory connectivity controls under an author-approved protocol, and to audit the manuscript's numbers against the machine-readable results. The authors take responsibility for all content.

REFERENCES

[1] H. Wang, P. Liu, J. Cheng, Z. Liang, J. Gu, Z. Li, Y. Ding, W. Jiang, Y. Shi, X. Qian, D. Z. Pan, F. T. Chong, and S. Han, "QuEst: Graph transformer for quantum circuit reliability estimation," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2022.

[2] M. Li, V. Shitole, E. Chien, C. Man, Z. Wang, S. Sridharan, Y. Zhang, T. Krishna, and P. Li, "LayerDAG: A layerwise autoregressive diffusion model for directed acyclic graph generation," in *International Conference on Learning Representations*, 2025.

[3] C. Beaudoin and S. Ghosh, "Q-Fusion: Diffusing quantum circuits," *arXiv preprint arXiv:2504.20794*, 2025.

[4] T. Fösel, M. Y. Niu, F. Marquardt, and L. Li, "Quantum circuit optimization with deep reinforcement learning," *arXiv preprint arXiv:2103.07585*, 2021.

[5] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2019, pp. 1001–1014.

[6] A. Cowtan, S. Dilkes, R. Duncan, A. Krajenbrink, W. Simmons, and S. Sivarajah, "On the qubit routing problem," in *14th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2019)*, ser. LIPIcs, vol. 135, 2019, pp. 5:1–5:32.

[7] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[8] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković, "Geometric deep learning: Grids, groups, graphs, geodesics, and gauges," *arXiv preprint arXiv:2104.13478*, 2021.

[9] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner *et al.*, "Relational inductive biases, deep learning, and graph networks," *arXiv preprint arXiv:1806.01261*, 2018.

[10] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. J. Smola, "Deep sets," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 3394–3404.

[11] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proceedings of the 34th International Conference on Machine Learning*, 2017, pp. 1263–1272.

[12] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations*, 2017.

[13] P. Veličković, R. Ying, M. Padovano, R. Hadsell, and C. Blundell, "Neural execution of graph algorithms," in *International Conference on Learning Representations (ICLR)*, 2020.

[14] G. Yehudai, E. Fetaya, E. Meirom, G. Chechik, and H. Maron, "From local structures to size generalization in graph neural networks," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 139, 2021, pp. 11 975–11 986.

[15] K. Xu, M. Zhang, J. Li, S. S. Du, K. Kawarabayashi, and S. Jegelka, "How neural networks extrapolate: From feedforward to graph neural networks," in *International Conference on Learning Representations (ICLR)*, 2021.

[16] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.

[17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 5998–6008.

[18] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," *arXiv preprint arXiv:1411.4028*, 2014.

[19] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien, "A variational eigenvalue solver on a photonic quantum processor," *Nature Communications*, vol. 5, p. 4213, 2014.